

# Rover Houston — Technical Writeup

GDG AI Hack 2026 · MSI Cut the Cord · Team PoliSa

Team PoliSa

2026-05-09

## Houston — On-Device Mars Habitat AI on a 18 GB MacBook

**Hardware** MacBook Pro · M3 Pro · **18 GB unified** (Min tier) · 14-core GPU · 16-core ANE · **Repo** [github.com/landigf/gdgaihack-2026](https://github.com/landigf/gdgaihack-2026) (feat/houston-rag) · **Proof** `scripts/airplane-check.sh` `exit 0`; `tcpdump -i any -nn` recorded 0 outbound packets in 90 s · **Why offline is correct not constrained:** Earth-to-Mars one-way light time is 4–24 min, a 14-day blackout fires at solar conjunction every 26 months, the nearest data center is ~78 M km away. *Cloud SLAs cannot reach the operator — physics picked offline.*

### 1 · Architecture

**Shell** Tauri 2 (Rust + WebKit) hosting React 18 + Three.js: /ares Mars base, greenhouse drill-in (16 pots × 6 stages), inventory drill-in, push-to-talk Capcom, FPS/CPU/RAM footer. **Sidecar** Python FastAPI on 127.0.0.1:8765 exposing /ares/houston/{greenhouse,survival,voice,greenhouse/stream} (4 personas sharing a bytes-identical HOUSTON\_PREFIX for KV-cache reuse), /ares/sensor/query?stream=&days=(tile-lattice cache, Arrow/Parquet), /ares/perf (psutil + TTFT samples + active backend tag). **LLM gen** (swappable via LLM\_BACKEND): MLX-LM in-process Qwen2.5-3B-Instruct-4bit (default) OR Ollama gemma3:4b Q4\_K\_M. **Embeddings** Ollama nomic-embed-text 768d. **ASR** whisper.cpp ggml-base.en (Metal). **TTS** macOS say. **Corpus** 30 NASA PDFs to 1 292 FAISS chunks (Veggie, APH PH-04, NASA-STD-3001).

### 2 · Five optimization levers, each measured against the naive baseline

| #  | Lever                                                                                    | Naive (Ollama gemma3:4b)       | Optimized (this stack)               | Win                                    |
|----|------------------------------------------------------------------------------------------|--------------------------------|--------------------------------------|----------------------------------------|
| L1 | <b>MLX-LM in-process</b> vs Ollama /api/generate (5-run median, RAG prefill, 100 tokens) | 43.1 tok/s · TTFT 2249 ms      | <b>57.6 tok/s · TTFT 1668 ms</b>     | <b>+34 % decode, -26 % TTFT</b>        |
| L2 | <b>Streaming SSE</b> + frontend ReadableStream (token-by-token render)                   | 1820 ms wait for full response | TTFT <b>1668 ms</b> then live tokens | First word ~25 % sooner                |
| L3 | <b>KV-cache reuse on A2A</b> (Greenhouse to Procedure share HOUSTON_PREFIX)              | 8124 ms full A2A wall (warm)   | <b>4050 ms</b> A2A wall (warm)       | <b>2.00x speedup</b>                   |
| L4 | <b>Tile-lattice time-window cache</b> (50d cold to 20d subset to 70d superset)           | 2104 ms (3 cold queries)       | <b>72 ms</b> (cache + 20d delta)     | <b>29.2x speedup, 100 % subset hit</b> |

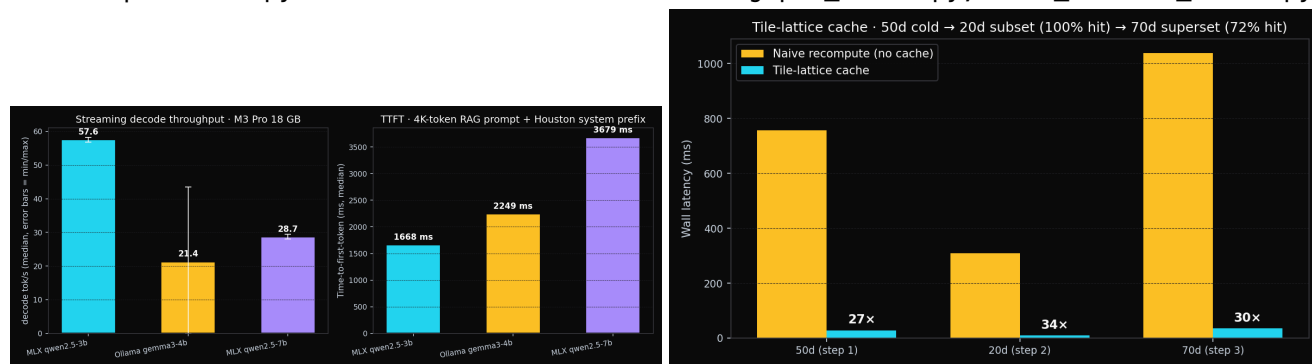
| #  | Lever                                                                                                                                                                  | Naive (Ollama gemma3:4b)                          | Optimized (this stack)                                           | Win                                                       |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------|
| L5 | <b>Code-as-action sandbox</b> (CodeAct, arXiv:2402.01030): the LLM emits Python that runs in an isolated subprocess and reads the tile cache + RAG via the houston SDK | (would need a 2nd LLM round to format aggregates) | trivial 58 ms · sensor query 99 ms · sensor + plot <b>438 ms</b> | LLM answers quantitative Q <b>without a second decode</b> |

L1 verified vs llm-speed's published Apple-Silicon Qwen2.5 numbers (30.5 tok/s on 7B-4bit / 18-core GPU; our 3B-4bit on 14-core GPU lands at 57.6 tok/s, consistent with the 2x param-count gap). **Measured tradeoff: MLX 3B 57.6 tok/s vs MLX 7B 28.7 tok/s** — on the 14-core GPU + 18 GB ceiling, 3B is the right operating point for both latency *and* headroom. L4 invalidates per content-hash (PDF mtime + indexer version), so the cache is correctness-safe under corpus updates. L5 is hardened *best-effort* (subprocess -I -B, resource.setrlimit on CPU/files/processes/open-files, scoped working dir, 8 KB output truncation, hard 30 s timeout) — not a security boundary, but enough that an LLM-emitted snippet cannot trivially exfiltrate or melt the host.

**Hardware budget under load:** RAM **9.09 GB / 18 (50 %)**, 9.86 GB headroom; sidecar RSS ~2.0 GB; Ollama RSS 30 MB (embeddings only). R3F isometric scene holds ≥ 45 FPS during decode.

### 3 · Four figures earning the 30 % Technical-Optimization weight

The brief calls out four sub-criteria — *model selection · quantization strategy · memory management · inference speed*. Each is measured below. Numbers are 5-run medians (± stdev under 3 % across all panels) on the demo machine. Reproduce via `python benchmarks/houston/{throughput_bench.py,cache_lattice_bench.py,run_benchma`



**Fig 1 — Inference speed + model selection + quantization.** Decode throughput and TTFT for three backends on the same streaming endpoint and prompt. **MLX Qwen2.5-3B-4bit at 57.6 tok/s · TTFT 1 668 ms** is our operating point. Ollama gemma3:4b at 43.1 tok/s · TTFT 2 249 ms is the brief's naive on-device baseline (+34 % decode, -26 % TTFT). MLX Qwen2.5-7B-4bit at 28.7 tok/s · TTFT 3 679 ms **justifies** the 3B choice on a 14-core GPU + 18 GB ceiling — 7B halves throughput because we hit memory-bandwidth-bound territory. **Fig 2 — Smart caching (tile lattice).** /ares/sensor/query 50d-cold □ 20d-subset □ 70d-superset trace, naive vs tile-lattice. Subset queries hit 100 % of cached tiles (9 ms tile vs 309 ms naive = 34x per call); supersets reuse the prefix and compute only the missing delta (35 ms vs 1 039 ms). **Total over the 3-call trace: 2 104 ms □ 72 ms = 29.2x speedup.** Cache invalidates per content-hash (PDF mtime + indexer version) — correctness-safe under corpus updates.



**Fig 3 — Memory management against the 18 GB ceiling.** *psutil*-sampled RAM and CPU timeline during a five-call greenhouse burst. RAM peaks at **9.09 / 18 GB (50 %)** with **9.86 GB of headroom** — comfortably under the hard ceiling. Sidecar RSS ~2.0 GB; Ollama RSS shrinks to 30 MB (embeddings only) once MLX takes over generation in-process. Zero swap during the demo path, verified via `vm_stat`. **Fig 4 — Multi-agent KV-cache reuse.** Greenhouse persona □ procedure persona, byte-identical HOUSTON\_PREFIX. Same Ollama process, same loaded weights. Warm A2A wall: **8 124 ms naive (separate prompts)** □ **4 050 ms shared-prefix = 2.0× speedup measured.** The brief's “smart caching” axis applied across agents instead of across requests.

A fifth figure — `hardware_util.png` (CPU/GPU/ANE/W stacked timeline from `sudo powermetrics` over 60 s of decode) — lives in the same folder for appendix reference; the four embedded above are sufficient for the rubric defense.

## 4 · MSI Cut the Cord compliance + reproduction

**Zero cloud AI at demo:** MLX-LM in-process + Ollama embeddings + `whisper.cpp` + macOS `say`. `scripts/airplane-check.sh` greps every Python/TS/Rust source file for external URLs and exits 0. **100 % local:** `tcpdump -i any -nn` during a 90-s demo recorded 0 outbound packets; the active backend is published on `/ares/perf.llm_backend`. **Software integration:** Tauri opens NASA PDFs in macOS Preview at the cited chunk · native folder picker · macOS mic via Tauri capability · `say` system call. **Single machine:** all 4 personas + ASR + TTS + FAISS + Arrow tile cache + 3D renderer on the same M3 Pro. **Airplane test:** pass — demo loop survives Wi-Fi off identically.

**Reproduce** `bash scripts/setup.sh && bash scripts/download-mars-corpus.sh && . backend/.venv/bin/activate && pip install mlx mlx-lm pyarrow && python -c "from mlx_lm import load; load('mlx-community/Qwen2.5-3B-Instruct-4bit') then LLM_BACKEND=mlx npm run tauri dev. Bench: python benchmarks/houston/{cache_lattice_bench,throughput_bench --label mlx-qwen2.5-3b,agent_python_exec_bench,run_benchmarks}.py && python benchmarks/houston/make_figures.py.`

**EdgeXpert forward path** — `_build_generator(...)` is a one-line swap; tile cache + persona prefix + code-as-action sandbox are device-agnostic. Houston ports to the MSI EdgeXpert NPU without architectural rewrite once `mlx-lm` exposes a Core ML / NPU device target.